Capítulo 06 - Agente Batedor (Scout Agent)

6.1. Introdução ao Agente Batedor

O **Agente Batedor** é o primeiro componente do pipeline do Projeto Quimera. Sua responsabilidade é simples, mas crítica: **descobrir e coletar URLs de perfis** de redes sociais (Instagram e Facebook) navegando por "hotspots digitais" pré-configurados.

6.1.1. O que é um Hotspot Digital?

Um **hotspot digital** é um perfil de rede social que atrai o público-alvo desejado. Exemplos:

- **Estabelecimentos:** Perfil do Instagram de um bar, restaurante, academia, café
- **Eventos:** Perfil de um evento cultural, festa, festival
- **Influenciadores:** Perfil de um influenciador local que o público-alvo segue

A lógica é simples: se você quer conhecer pessoas que frequentam academias em Varginha, navegue até o perfil do Instagram da academia mais popular da cidade e colete os perfis de quem segue, curte ou comenta nas postagens.

6.1.2. Responsabilidades do Agente Batedor

1. **Ler lista de hotspots** ativos do banco de dados
2. **Navegar até cada hotspot** usando um navegador automatizado (Selenium)
3. **Extrair URLs de perfis** de:
   - Seguidores do hotspot
   - Pessoas que curtiram postagens recentes
   - Pessoas que comentaram em postagens recentes
4. **Dedplicar URLs** (remover duplicatas)
5. **Armazenar URLs** no banco de dados com status `descoberto`
6. **Emitir evento** `PerfilColetado` para o próximo agente (Analista)

6.1.3. Desafios Principais

1. Detecção de Automação

Instagram e Facebook possuem sistemas sofisticados de detecção de bots. Se o sistema detectar que você está usando automação, sua conta será banida. O Agente Batedor precisa **simular comportamento humano** perfeitamente.

2. Rate Limiting

Plataformas limitam o número de ações por tempo. Fazer 1000 requisições em 1 minuto resultará em bloqueio. Precisamos **respeitar limites** e adicionar delays realistas.

3. Mudanças na Interface

Instagram e Facebook mudam frequentemente a estrutura HTML de suas páginas. O código de scraping precisa ser **robusto e adaptável**.

6.2. Arquitetura do Agente Batedor

6.2.1. Componentes Principais

Plain Text

```
┌──────────────────────────────────────────────────────────────┐
│                AGENTE BATEDOR                                 │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   ┌──────────────────────────────────────────┐               │
│   │  1. GERENCIADOR DE SESSÕES               │               │
│   │  - Inicializa navegador                  │               │
│   │  - Gerencia cookies e sessões            │               │
│   │  - Rotação de proxies                    │               │
│   │  - Rotação de User-Agents                │               │
│   └──────────────────────────────────────────┘               │
│                       │                                       │
│                       ▼                                       │
│   ┌──────────────────────────────────────────┐               │
│   │  2. NAVEGADOR (Selenium/Playwright)      │               │
│   │  - Chrome com undetected-chromedriver    │               │
│   │  - Simula comportamento humano           │               │
│   │  - Executa ações (scroll, click, wait)   │               │
│   └──────────────────────────────────────────┘               │
│                       │                                       │
│                       ▼                                       │
│   ┌──────────────────────────────────────────┐               │
│   │  3. EXTRATOR DE PERFIS                   │               │
│   │  - Extrai seguidores                     │               │
│   │  - Extrai curtidas                       │               │
│   │  - Extrai comentários                    │               │
│   │  - Parse de HTML                         │               │
│   └──────────────────────────────────────────┘               │
│                       │                                       │
│                       ▼                                       │
│   ┌──────────────────────────────────────────┐               │
│   │  4. DEDUPLICADOR                         │               │
```

```
│  │  - Remove URLs duplicadas                │           │
│  │  - Verifica se já existe no BD           │           │
│  └──────────────────────────────────────────┘           │
│                      │                                  │
│                      ▼                                  │
│  ┌──────────────────────────────────────────┐           │
│  │  5. PERSISTÊNCIA                         │           │
│  │  - Salva no PostgreSQL                   │           │
│  │  - Emite evento para Redis               │           │
│  └──────────────────────────────────────────┘           │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

6.2.2. Fluxo de Execução

Plain Text

```
INÍCIO
  │
  ├─> Ler hotspots ativos do banco de dados
  │
  ├─> Para cada hotspot:
  │     │
  │     ├─> Inicializar sessão do navegador
  │     │     ├─> Selecionar proxy aleatório
  │     │     ├─> Selecionar User-Agent aleatório
  │     │     ├─> Configurar fingerprint randomizado
  │     │     └─> Abrir navegador Chrome
  │     │
  │     ├─> Navegar até o perfil do hotspot
  │     │     ├─> Aguardar carregamento da página
  │     │     ├─> Verificar se página carregou corretamente
  │     │     └─> Simular leitura (delay 2-5 segundos)
  │     │
  │     ├─> Extrair seguidores (até 500)
  │     │     ├─> Clicar em "Seguidores"
  │     │     ├─> Aguardar modal abrir
  │     │     ├─> Scroll gradual para carregar mais
  │     │     ├─> Extrair URLs de cada perfil
  │     │     └─> Fechar modal
  │     │
  │     ├─> Extrair curtidas de postagens (até 200)
  │     │     ├─> Navegar até postagens recentes
  │     │     ├─> Para cada postagem:
  │     │     │     ├─> Clicar em "Curtidas"
  │     │     │     ├─> Extrair URLs
  │     │     │     └─> Voltar
```

```
    |      |      └─> Limitar a 3 postagens
    |      |
    |      ├─> Extrair comentários (até 100)
    |      |      ├─> Para cada postagem:
    |      |      |      ├─> Expandir comentários
    |      |      |      ├─> Extrair URLs de quem comentou
    |      |      |      └─> Voltar
    |      |      └─> Limitar a 3 postagens
    |      |
    |      ├─> Deduplicar URLs coletadas
    |      |
    |      ├─> Verificar quais URLs já existem no banco
    |      |
    |      ├─> Salvar novas URLs no PostgreSQL
    |      |
    |      ├─> Emitir evento PerfilColetado para cada URL
    |      |
    |      ├─> Atualizar métricas do hotspot
    |      |
    |      └─> Fechar navegador
    |
    └─> Aguardar próximo ciclo (1 hora)
    |
  FIM
```

6.3. Técnicas de Web Scraping

6.3.1. Selenium vs. Playwright

Selenium:

- ✅ Maduro e estável
- ✅ Grande comunidade
- ✅ Suporte a `undetected-chromedriver` (bypass de detecção)
- ❌ Mais lento que Playwright
- ❌ API menos moderna

Playwright:

- ✅ Mais rápido e confiável
- ✅ API moderna e assíncrona
- ✅ Melhor handling de SPAs (Single Page Applications)

- ❌ Menos ferramentas de anti-detecção

Decisão: Usamos **Selenium com undetected-chromedriver** pela capacidade superior de evitar detecção.

6.3.2. Seletores CSS e XPath

Para extrair dados de páginas HTML, usamos **seletores CSS** ou **XPath**.

Exemplo de estrutura HTML do Instagram:

```HTML
<div class="x1dm5mii">
  <a href="/username/" class="_acan">
    <span class="_ap3a">Nome do Usuário</span>
  </a>
</div>
```

Seletor CSS:

```Python
elementos = driver.find_elements(By.CSS_SELECTOR, "a._acan")
for elemento in elementos:
    url = elemento.get_attribute("href")
    print(url)  # /username/
```

XPath:

```Python
elementos = driver.find_elements(By.XPATH, "//a[@class='_acan']")
```

Dica: Instagram usa classes CSS geradas automaticamente (como `x1dm5mii`, `_acan`) que podem mudar. Use seletores mais genéricos ou múltiplos seletores como fallback.

6.3.3. Waiting Strategies

Problema: Páginas modernas (SPAs) carregam conteúdo dinamicamente via JavaScript. Se tentarmos extrair dados antes do carregamento completo, não encontraremos nada.

Solução: Usar **waits explícitos**.

Tipos de Wait:

1. **Implicit Wait** (não recomendado)

```python
driver.implicitly_wait(10)  # Espera até 10s por qualquer elemento
```

1. **Explicit Wait** (recomendado)

```python
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# Esperar até que o elemento esteja presente
wait = WebDriverWait(driver, 10)
elemento = wait.until(
    EC.presence_of_element_located((By.CSS_SELECTOR, "a._acan"))
)
```

1. **Custom Wait** (mais flexível)

```python
import time

def esperar_elemento(driver, seletor, timeout=10):
    """Espera até que elemento apareça ou timeout."""
    tempo_inicio = time.time()
    while time.time() - tempo_inicio < timeout:
        try:
            elemento = driver.find_element(By.CSS_SELECTOR, seletor)
            return elemento
        except:
            time.sleep(0.5)
    raise TimeoutError(f"Elemento {seletor} não encontrado após {timeout}s")
```

6.3.4. Scroll Infinito

Instagram e Facebook usam **scroll infinito**: mais conteúdo é carregado conforme você scrolla a página.

Técnica de Scroll:

```python
def scroll_gradual(driver, distancia=300, delay=1.0):
    """Scrolla a página gradualmente, simulando humano."""
    # Obter altura atual da página
    altura_anterior = driver.execute_script("return
```

```
document.body.scrollHeight")

    # Scroll gradual
    driver.execute_script(f"window.scrollBy(0, {distancia});")
    time.sleep(delay)

    # Obter nova altura
    altura_nova = driver.execute_script("return document.body.scrollHeight")

    # Retornar True se carregou mais conteúdo
    return altura_nova > altura_anterior

# Uso: scroll até carregar 500 perfis
perfis_carregados = 0
while perfis_carregados < 500:
    if not scroll_gradual(driver):
        break  # Não há mais conteúdo
    perfis_carregados = len(driver.find_elements(By.CSS_SELECTOR, "a._acan"))
```

6.4. Estratégias de Anti-Detecção

6.4.1. Como Plataformas Detectam Bots

Sinais de Detecção:

1. **WebDriver Flag:** Navegadores automatizados expõem `navigator.webdriver = true`
2. **Ausência de Plugins:** Navegadores reais têm plugins (Flash, PDF Viewer)
3. **Canvas Fingerprinting:** Renderização de canvas é única por dispositivo
4. **Movimentos de Mouse:** Bots não movem o mouse ou movem de forma robótica
5. **Velocidade de Ações:** Bots executam ações muito rápido
6. **User-Agent Inconsistente:** User-Agent não bate com características do navegador
7. **IP Suspeito:** IP de datacenter em vez de residencial

6.4.2. Técnica 1: undetected-chromedriver

undetected-chromedriver é uma versão modificada do ChromeDriver que remove sinais de automação.

O que ele faz:

- Remove `navigator.webdriver`
- Adiciona plugins falsos

- Modifica propriedades do navegador
- Randomiza fingerprints

Instalação:

```bash
pip install undetected-chromedriver
```

Uso básico:

```python
import undetected_chromedriver as uc

# Criar driver "indetectável"
driver = uc.Chrome()

# Navegar normalmente
driver.get("https://instagram.com")
```

6.4.3. Técnica 2: Rotação de Proxies

Por que usar proxies?

Se você fizer 1000 requisições do mesmo IP, o Instagram saberá que é um bot. Proxies residenciais fazem parecer que as requisições vêm de usuários diferentes.

Tipos de Proxy:

1. **Datacenter Proxies:** Baratos ($1-2/GB), mas facilmente detectados
2. **Residential Proxies:** Caros ($5-15/GB), mas indistinguíveis de IPs reais
3. **Mobile Proxies:** Muito caros ($30+/GB), IPs de operadoras móveis

Recomendação: Usar **residential proxies** de provedores como IPRoyal, Webshare ou Smartproxy.

Configuração:

```python
import undetected_chromedriver as uc

# Configurar proxy
proxy = "http://usuario:senha@proxy.iproyal.com:12321"

options = uc.ChromeOptions()
```

```
options.add_argument(f'--proxy-server={proxy}')

driver = uc.Chrome(options=options)
```

Rotação de Proxies:

Python

```
proxies = [
    "http://usuario:senha@proxy1.iproyal.com:12321",
    "http://usuario:senha@proxy2.iproyal.com:12322",
    "http://usuario:senha@proxy3.iproyal.com:12323",
]

import random

def obter_proxy_aleatorio():
    return random.choice(proxies)

# Usar proxy diferente a cada sessão
proxy = obter_proxy_aleatorio()
options.add_argument(f'--proxy-server={proxy}')
```

6.4.4. Técnica 3: Rotação de User-Agents

User-Agent identifica o navegador e sistema operacional. Bots costumam usar o mesmo User-Agent sempre.

Lista de User-Agents Reais:

Python

```
USER_AGENTS = [
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/120.0.0.0 Safari/537.36",
    "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36",
    "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/120.0.0.0 Safari/537.36",
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:121.0) Gecko/20100101
Firefox/121.0",
    "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15
(KHTML, like Gecko) Version/17.1 Safari/605.1.15",
]

import random
```

```python
def obter_user_agent_aleatorio():
    return random.choice(USER_AGENTS)

# Configurar User-Agent
user_agent = obter_user_agent_aleatorio()
options.add_argument(f'--user-agent={user_agent}')
```

6.4.5. Técnica 4: Randomização de Delays

Problema: Bots executam ações em intervalos regulares (ex: exatamente 1 segundo entre cliques). Humanos são irregulares.

Solução: Adicionar delays aleatórios.

Python

```python
import random
import time

def delay_humano(min_segundos=2, max_segundos=5):
    """Aguarda um tempo aleatório entre min e max."""
    tempo = random.uniform(min_segundos, max_segundos)
    time.sleep(tempo)

# Uso
driver.get("https://instagram.com/perfil")
delay_humano(2, 5)  # Aguarda 2-5 segundos
driver.find_element(By.CSS_SELECTOR, "button").click()
delay_humano(1, 3)  # Aguarda 1-3 segundos
```

6.4.6. Técnica 5: Simulação de Movimentos de Mouse

Problema: Bots não movem o mouse. Humanos movem o mouse constantemente.

Solução: Usar biblioteca `pyautogui` ou executar JavaScript para mover o cursor.

Python

```python
from selenium.webdriver.common.action_chains import ActionChains
import random

def mover_mouse_aleatorio(driver):
    """Move o mouse para uma posição aleatória da tela."""
    largura = driver.execute_script("return window.innerWidth")
    altura = driver.execute_script("return window.innerHeight")

    x = random.randint(0, largura)
```

```
    y = random.randint(0, altura)

    # Mover mouse usando ActionChains
    action = ActionChains(driver)
    action.move_by_offset(x, y).perform()

    # Resetar posição do mouse
    action.move_by_offset(-x, -y).perform()

# Uso
mover_mouse_aleatorio(driver)
delay_humano(0.5, 1.5)
```

6.4.7. Técnica 6: Fingerprint Randomization

Canvas Fingerprinting: Sites podem identificar seu navegador pela forma como ele renderiza gráficos.

Solução: Injetar JavaScript para randomizar canvas.

Python

```
def randomizar_canvas(driver):
    """Injeta script para randomizar canvas fingerprint."""
    script = """
    const originalToDataURL = HTMLCanvasElement.prototype.toDataURL;
    HTMLCanvasElement.prototype.toDataURL = function() {
        // Adicionar ruído aleatório ao canvas
        const context = this.getContext('2d');
        const imageData = context.getImageData(0, 0, this.width,
this.height);
        for (let i = 0; i < imageData.data.length; i += 4) {
            imageData.data[i] += Math.random() * 10 - 5;
        }
        context.putImageData(imageData, 0, 0);
        return originalToDataURL.apply(this, arguments);
    };
    """
    driver.execute_cdp_cmd("Page.addScriptToEvaluateOnNewDocument", {
        "source": script
    })

# Executar antes de navegar
randomizar_canvas(driver)
driver.get("https://instagram.com")
```

6.5. Limites e Rate Limiting

6.5.1. Limites do Instagram

Limites Conhecidos (não oficiais):

Ação	Limite por Hora	Limite por Dia
Seguir usuários	60	200
Curtir postagens	60	350
Comentar	30	100
Enviar mensagens	20	50
Visualizar perfis	200	1000

Consequências de Exceder Limites:

1. **Soft Ban:** Ação bloqueada temporariamente (1-24 horas)
2. **Shadowban:** Conta fica invisível para outros usuários
3. **Hard Ban:** Conta permanentemente suspensa

6.5.2. Estratégia de Rate Limiting

Regras:

1. **Limite de ações por sessão:** Máximo 200 ações (scroll, click, extração)
2. **Intervalo entre ações:** 2-5 segundos (aleatório)
3. **Intervalo entre sessões:** 10-30 minutos
4. **Limite diário:** Máximo 500 perfis coletados por conta

Implementação:

Python

```python
class RateLimiter:
    def __init__(self, max_acoes_por_sessao=200, max_acoes_por_dia=500):
        self.max_acoes_por_sessao = max_acoes_por_sessao
        self.max_acoes_por_dia = max_acoes_por_dia
        self.acoes_sessao = 0
        self.acoes_dia = 0
        self.timestamp_inicio_sessao = time.time()
```

```python
    def pode_executar_acao(self):
        """Verifica se pode executar mais uma ação."""
        if self.acoes_sessao >= self.max_acoes_por_sessao:
            return False
        if self.acoes_dia >= self.max_acoes_por_dia:
            return False
        return True

    def registrar_acao(self):
        """Registra que uma ação foi executada."""
        self.acoes_sessao += 1
        self.acoes_dia += 1

    def resetar_sessao(self):
        """Reseta contador de sessão."""
        self.acoes_sessao = 0
        self.timestamp_inicio_sessao = time.time()

# Uso
limiter = RateLimiter()

for hotspot in hotspots:
    if not limiter.pode_executar_acao():
        print("Limite atingido. Aguardando...")
        time.sleep(1800)  # 30 minutos
        limiter.resetar_sessao()

    # Executar ação
    coletar_perfis(hotspot)
    limiter.registrar_acao()
```

6.6. Tratamento de Erros e Resiliência

6.6.1. Tipos de Erros

1. Erros de Rede:

- Timeout de conexão
- Proxy indisponível
- Página não carrega

2. Erros de Scraping:

- Elemento não encontrado

- Estrutura HTML mudou
- Modal não abre

3. Erros de Detecção:

- Captcha aparece
- Conta bloqueada temporariamente
- Página de verificação

6.6.2. Estratégia de Retry

Exponential Backoff: Aguardar tempo crescente entre tentativas.

Python

```python
import time

def executar_com_retry(funcao, max_tentativas=5, backoff_inicial=1):
    """Executa função com retry e exponential backoff."""
    for tentativa in range(max_tentativas):
        try:
            return funcao()
        except Exception as e:
            if tentativa == max_tentativas - 1:
                raise e  # Última tentativa, propagar erro

            tempo_espera = backoff_inicial * (2 ** tentativa)
            print(f"Erro: {e}. Tentando novamente em {tempo_espera}s...")
            time.sleep(tempo_espera)

# Uso
def navegar_perfil():
    driver.get("https://instagram.com/perfil")
    return driver.find_element(By.CSS_SELECTOR, "h1").text

nome = executar_com_retry(navegar_perfil)
```

6.6.3. Circuit Breaker

Se um hotspot falha repetidamente, marcá-lo como "problemático" e pular temporariamente.

Python

```python
class CircuitBreaker:
    def __init__(self, threshold=5, timeout=3600):
```

```
        self.threshold = threshold  # Número de falhas para abrir circuito
        self.timeout = timeout  # Tempo em segundos para tentar novamente
        self.falhas = {}
        self.circuitos_abertos = {}

    def registrar_falha(self, hotspot_id):
        """Registra uma falha para um hotspot."""
        self.falhas[hotspot_id] = self.falhas.get(hotspot_id, 0) + 1

        if self.falhas[hotspot_id] >= self.threshold:
            self.circuitos_abertos[hotspot_id] = time.time()
            print(f"Circuito aberto para hotspot {hotspot_id}")

    def pode_tentar(self, hotspot_id):
        """Verifica se pode tentar acessar o hotspot."""
        if hotspot_id not in self.circuitos_abertos:
            return True

        tempo_abertura = self.circuitos_abertos[hotspot_id]
        if time.time() - tempo_abertura > self.timeout:
            # Timeout expirou, resetar
            del self.circuitos_abertos[hotspot_id]
            self.falhas[hotspot_id] = 0
            return True

        return False

# Uso
breaker = CircuitBreaker()

for hotspot in hotspots:
    if not breaker.pode_tentar(hotspot.id):
        print(f"Pulando hotspot {hotspot.id} (circuito aberto)")
        continue

    try:
        coletar_perfis(hotspot)
    except Exception as e:
        breaker.registrar_falha(hotspot.id)
```

6.7. Métricas e Monitoramento

6.7.1. Métricas Importantes

Métricas de Performance:

- Perfis coletados por hora
- Tempo médio por hotspot
- Taxa de sucesso (%)

Métricas de Qualidade:

- Taxa de duplicação (%)
- Taxa de perfis válidos (%)

Métricas de Saúde:

- Taxa de erro (%)
- Número de retries
- Circuitos abertos

6.7.2. Logging Estruturado

Exemplo de log:

JSON

```
{
  "timestamp": "2025-10-10T16:30:00Z",
  "nivel": "INFO",
  "agente": "agente_batedor",
  "acao": "coletar_perfis",
  "hotspot_id": "uuid-123",
  "hotspot_nome": "Academia Varginha",
  "perfis_coletados": 87,
  "tempo_segundos": 45.3,
  "proxy_usado": "proxy1.iproyal.com",
  "sucesso": true
}
```

6.8. Boas Práticas

6.8.1. Respeitar Robots.txt

Embora estejamos fazendo scraping de forma não autorizada, é importante conhecer as regras:

Plain Text

```
User-agent: *
Disallow: /accounts/
Disallow: /api/
```

6.8.2. Minimizar Impacto

- Não sobrecarregar servidores (delays entre requisições)
- Usar cache quando possível
- Limitar número de requisições

6.8.3. Ética

- Não coletar dados sensíveis além do necessário
- Não vender dados coletados
- Respeitar quando alguém pede para parar

6.9. Conclusão do Capítulo

Este capítulo cobriu o **Agente Batedor** em profundidade:

- **Responsabilidades:** Coletar URLs de perfis de hotspots
- **Técnicas de Scraping:** Selenium, seletores, waits, scroll
- **Anti-Detecção:** undetected-chromedriver, proxies, User-Agents, delays, fingerprinting
- **Rate Limiting:** Limites do Instagram, estratégias de throttling
- **Resiliência:** Retry, exponential backoff, circuit breaker
- **Monitoramento:** Métricas, logging estruturado

No próximo capítulo, veremos o **Agente Analista**, que recebe os perfis coletados e os analisa usando NLP e Computer Vision.

Próximo Capítulo: Capítulo 07 - Agente Analista

Referências:

- Selenium Documentation: https://www.selenium.dev/documentation/
- undetected-chromedriver: https://github.com/ultrafunkamsterdam/undetected-chromedriver
- Instagram Rate Limits: https://www.inflact.com/blog/instagram-limits/

- Web Scraping Best Practices: https://www.scraperapi.com/blog/web-scraping-best-practices/